

# ClaudeForth — Project Summary (Starting Point for New Project)

## What this is

ClaudeForth is a 6809 assembly-language implementation of ANS Forth, targeting a MAME-emulated 6809 system (the `mecb_6809` profile). It prioritizes ANS standards compliance alongside practical debuggability. The project is currently in an active runtime bug-fixing phase: manual testing of the word glossary against real MAME execution, driven by single-stepping through the MAME debugger.

This document summarizes the technical state built up across an extended prior conversation, so a new conversation/project can pick up from here without re-deriving it.

## Deliverable files (attach these to the new project's knowledge base)

- `forth6809.asm` — the consolidated, current source (~6,400 lines). This is the single source of truth.
- `forth6809_split/` — the same source split into 28 files by section, for easier review. Files `01_vectors.asm` through `27_forth_dictionary.asm` correspond to numbered `SECTION N:` markers in the consolidated file; `00_memory_map_and_globals.asm` is the preamble before the first section marker (memory-map constants, `GLOBALS` layout, and — as of the most recent work — the unit test framework, which lives physically in that same region).
- `OPEN_ITEMS_CHECKLIST.md` — a running, append-only log of every finding, fix, and still-open question, in chronological order. Treat this as authoritative history — entries are never rewritten, only marked resolved/superseded with a note, so the log stays honest about what was believed at each point in time.
- `ClaudeForth.docx` (+ PDF preview) — formal documentation: memory map, glossary, architecture (including a UML activity diagram of the outer interpreter), build instructions, and the full assembler source reproduced as an appendix.

## Architecture (established, stable)

- **Threading model:** subroutine-threaded (STC) — `JSR / RTS` is the inner interpreter.
- **Registers:** `U` = data stack ( `PSHU / PULU` ), `S` = return stack, `X / Y` = scratch, `D` = TOS arithmetic. Cell size is 16-bit.
- **Dictionary:** 224 entries, chained via `LINK` fields. `BASELATEST` (currently `H_M2` ) marks the true chain head; `COLD` initializes the runtime `LATEST` variable from it. The chain terminates at `H_KEY` with a `LINK` of `0` .
- **Header naming convention:** every dictionary header is `H_<NAME>` , e.g. `H_DUP` , `H_QUIT` . CFA is normally the primitive's own code label directly (no trampoline); `CONSTANT / VALUE` -pattern words use a `DODOES` -trampoline instead.

## Current memory map (approximate — check

`00_memory_map_and_globals.asm` for exact live values, since several regions have moved multiple times)

|                                                                                                                                                                         |                       |                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------------------------------------------------------------------------------|
| <code>\$FFF0-\$FFFF</code>                                                                                                                                              | <code>VECTORS</code>  | Hardware vectors                                                                                              |
| <code>\$FFA9-\$FFEF</code>                                                                                                                                              | <code>INITCODE</code> | <code>COLDSTRT/WARM</code> (71 bytes real, assembler-confirmed)                                               |
| <code>\$E02A-...</code>                                                                                                                                                 | <code>BASECODE</code> | ROM primitives (8110-byte nominal budget)                                                                     |
| <code>\$D83F-\$E006</code>                                                                                                                                              | <code>BASEDICT</code> | ROM dictionary headers (1992 bytes)                                                                           |
| <code>\$C100-...</code>                                                                                                                                                 |                       | (unit test framework, when <code>UNITTESTS=0</code> ) then <code>FILL</code> padding to <code>BASEDICT</code> |
| <code>\$C000-\$C0FF</code>                                                                                                                                              | <code>INOUT</code>    | I/O block; <code>ACIA</code> at <code>INOUT+8</code>                                                          |
| <code>\$BD00-\$BFFF</code>                                                                                                                                              | <code>RSTACK</code>   | Return stack ( <code>S</code> ), grows down                                                                   |
| <code>\$B900-\$BCFF</code>                                                                                                                                              | <code>DSTACK</code>   | Data stack ( <code>U</code> ), grows down                                                                     |
| <code>\$7000-\$B8FF</code>                                                                                                                                              | <code>APPCODE</code>  | Application code                                                                                              |
| <code>\$2000-\$6EA4</code>                                                                                                                                              | <code>APPDICT</code>  | Application dictionary                                                                                        |
| <code>\$021B-\$1FFF</code>                                                                                                                                              | <code>APPVARS</code>  | Mutable variable space                                                                                        |
| (below <code>\$021B</code> : <code>TIBBUF</code> , <code>WORDBUF</code> , <code>SIBUF</code> , and other fixed buffers, then <code>GLOBALS</code> at <code>\$0</code> ) |                       |                                                                                                               |

**Known, currently-unresolved memory-map issues** (see the checklist for exact figures — these numbers have shifted several times and the checklist has the current ones):

- A real overlap between `INITCODE` and `BASECODE` 's *nominal* budget (not necessarily its real content, which is smaller and unconfirmed by a full assembler run).
- `FILL` 's status as a genuine LWASM directive has never been independently confirmed.
- These are tracked, not blocking — they haven't caused any of the runtime bugs found so far.

## Conditional assembly conventions

Two independent flags, both following the same pattern:

```
SERIALPOLL EQU 1    ; 1 = polling I/O (active default); 0 = interrupt-driven
UNITTESTS  EQU 0    ; 0 = unit test framework included; 1 = excluded
```

Convention: `IFEQ <FLAG>` branches are taken when the flag **equals zero**. This is *not* intuitive at a glance ( `SERIALPOLL=1` is the active default but takes the `ELSE` branch) — always check the actual flag value rather than assume from the block's position. Visual markers used throughout: `; >>>>>>>>> on IFEQ , ; <<<<<<<<<< on ELSE , ; <<<<<<<<<< on ENDC .`

`SERIALPOLL` gates three blocks: `IRQH` (interrupt handler vs. bare `RTI`), `KEY / KEYQ / EMIT` (buffered+interrupt vs. direct polling), and `COLDSTRT`'s ACIA control-register init. **Important finding from this session:** interrupt-driven mode never touches the data stack ( `U` ) at all — this was used to rule out “interrupt mode masks the bug” hypotheses during debugging.

## The core debugging methodology established this session

**Static source tracing is not reliable enough on its own for this codebase.** Across many turns, careful manual register-by-register tracing of `WORD`, `FIND`, `EXECUTE`, `CCALL`, `DOTEST`, `LOOP`, and others repeatedly concluded “this looks correct” — and was wrong, sometimes in ways that took several more turns to actually locate. Every one of the real bugs found this session was ultimately confirmed (and several were *only found at all*) via actual MAME debugger single-stepping, not static analysis. Several bugs only became *reachable* — and thus visible — once an earlier bug stopped masking them; fixing one bug in this codebase reliably exposed the next one underneath it.

**Practical implication for a new conversation:** treat any “I traced this and it looks correct” conclusion as provisional. When something is reported as broken on real hardware, prioritize getting real debugger register/memory state over further static tracing, and be willing to say “I can’t find this through source reading alone” rather than force a conclusion.

## Complete bug history (chronological, all confirmed via MAME debugger unless noted)

### 1. `WORD` — stranded stack address ( `IDONE` fix)

`WORD`'s `EMPTY` branch always pushes a `c-addr` (matching its normal contract), but `INTERPRET` only *peeked* at it ( `LDX`, `U`, never popped) before branching to `IDONE`, which was a bare `RTS`. Left a stranded address on `U` at the end of every line. Fixed by making

IDONE do PULU X / RTS , matching the convention FIND 's own PULU X already used.

## 2. WORD — length corruption (register clobber)

WORD stored the wrong length byte for any token followed by more input on the same line. Root cause: B is D 's low byte on the 6809; SCANLP 's INCB loop counted the true length into B , but the subsequent TFR X,D (part of computing >IN ) silently destroyed it before STB ,X+ stored it — so the stored “length” was actually >IN 's delta (which happens to include the consumed trailing delimiter). This is why it only affected the first N-1 tokens on a line, never the last (the last token is terminated by running out of buffer, not by consuming a delimiter, so its >IN delta happens to equal its true length by coincidence — which is exactly why every earlier single-token test never surfaced this). Fixed by saving/restoring B ( PSHS B / PULS B ) around the >IN computation.

## 3. CCALL — corrupted compiled call targets

CCALL compiles a JSR to a given execution token — used by every colon definition for every word it contains. PULU D pulled the target address; LDA #OPJSR (loading the opcode to compile) then overwrote A , which is D 's high byte, silently destroying the top byte of the address just pulled. STD ,X++ then wrote the corrupted address out. This affected every compiled call, unconditionally — : tv . ; failed with “attempted execution of random memory.” Fixed by deferring PULU D until after STA ,X+ , so D is never live at the same time A gets reused.

## 4. CONSTANT — self-referential PFA

1234 CONSTANT c1 then executing c1 returned the compile-time address instead of 1234 . LDD CODEHERE read the address of the PFA field being written *before* it was written (self-referential), rather than the address 2 bytes further on where the value actually landed. Fixed with ADDD #2 . ( VALUEW , the structurally similar routine, was checked and confirmed *not* to share this bug — its PFA points into VARHERE , a separate region, not into itself.)

## 5–7. DO / I / LOOP — three distinct root causes, twelve total fix sites

The largest concentration of bugs in the session, found in sequence as each one unmasked the next:

- **DOTEST return-stack offset bug:** : lpy 10 3 DO I . LOOP ; exited after one pass. PULS X ran before DOTEST 's fixed-offset reads ( 2,S / 4,S / 6,S — the index/limit/leave-flag DOSETUP had pushed), shifting the stack by 2 first, so the leave-flag check read a leftover return address instead (almost always nonzero → false exit). The identical bug lived in LEAVE and DOPLUSTEST ( +LOOP 's runtime) — **3 sites**, one root cause: defer the

pop until each path actually needs it.

- **Branch-target-clobbering bug:** once loop bodies could actually execute more than once, `LOOP` compiled a branch displacement of `+8` instead of `-8`. Root cause: `LOOP` saved its backward-branch target in `X` via `PULU X`, then called `CCALL / CODECOMMA` — both of which internally reload `X` for their own purposes ( `LDX CODEHERE` ), silently destroying the saved target before `TFR X,D` read it back. The same pattern was found (by systematically sweeping every `PULU X` in the file, not just the reported case) in `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF` — **7 sites**. Two different fixes depending on structure: parking the target on `U` itself where nothing else touches `U` in between ( `LOOP / +LOOP / UNTIL / AGAIN / REPEAT` ), or a scratch variable where something else ( `CODEHERE` ) gets pushed onto `U` in between ( `ELSE / ENDOF` — an initial attempt incorrectly reused the park-on-`U` approach here and was caught and corrected before delivery).
- **IWORD / JWORD bug:** `I` printed the loop *limit* instead of the *index* on every iteration. `PULS X / PSHS X` bracketed the fixed-offset read for no reason (nothing needed offset 0), shifting `S` by 2 first so `LDD 2,S` read what was really at `4,S`. **2 sites** ( `I` and `J` ), identical fix: remove the unnecessary pop/push pair.

After all three were fixed, `: lpy 10 3 DO I . LOOP ;` was confirmed producing correct output on real MAME.

## New capability added: assembly-level unit test framework (not yet run on real hardware)

Gated by `UNITTESTS EQU 0/1`, matching the `SERIALPOLL` convention. `JSR TSTRUNNER` is called at the true end of `COLDSTRT` (after `INITSERIAL`, before `JMP COLD` — at that point `U / S` are valid but nothing else is initialized yet). The framework itself lives at `USROMSTRT` (`$C100`), previously unused `FILL`-padded ROM space; `UNITTESTS=1` reverts that region to exactly the padding it was before, computed automatically ( `FILL $FF,BASEDICT-ROM` ) so it stays correct as the test code's size changes.

Design principles: each test is independent by construction — it saves `U` before touching anything and *unconditionally* restores it at the end regardless of pass/fail, so a failed assertion can't leave stack residue for the next test. Test scratch variables borrow the start of `APPVARS` (safe only because tests run strictly before `COLD` re-initializes that address — a constraint worth remembering if the call site ever moves). Reporting is shared via `TSTREPORT`: prints the test's name (a counted string, via `COUNT + TYPE` — the same mechanism `BADWORD` uses to print a failing word), then `" OK"` or `" FAIL"`, then a `CR`.

`TSTDUP` is the only test written so far. It verifies `DUP` 's stack contents (duplicate and original both match a deliberately non-trivial test value, not `0 / 1 / -1` ; a sentinel pushed beneath is confirmed undisturbed) and its exact stack-pointer movement (2 bytes, measured immediately around the `JSR DUP` itself, isolated from the setup pushes).

`TSTRUNNER` / `TSTSTACK` are structured as extensible dispatcher lists for adding more tests/groups later.

**This framework has been verified structurally** (assembles cleanly across all four `SERIALPOLL` × `UNITTESTS` combinations, zero duplicate symbols, dictionary chain intact) **but not yet exercised on real MAME**. Given everything above about the gap between clean static analysis and real execution, treat it as unverified until it's actually been run.

## Working conventions for whoever continues this project

- **Every source change** gets: (1) applied to `forth6809.asm` , (2) verified via a duplicate-symbol/dictionary-chain-walk simulation across all conditional-assembly combinations, (3) propagated to the 28 split files with a byte-exact reassembly check against the consolidated source, (4) logged in `OPEN_ITEMS_CHECKLIST.md` .
- **The checklist is a historical log, not a living document** — past entries are never rewritten to look retroactively correct. When something is superseded, a new entry says so explicitly and points back to the old one, which stays as-is.
- **When a fix is requested based on the reporter's own analysis**, that analysis gets independently verified against the actual code (traced line-by-line) before being confirmed — not rubber-stamped, and not silently second-guessed either. Several turns this session involved explicitly confirming or gently correcting a proposed diagnosis with the specific code trace as evidence.
- **Docx documentation** is maintained via direct XML manipulation (python-docx) rather than a lost original generator script — matching Word's own heading/TOC bookmark conventions carefully (existing bookmark IDs, `PAGEREF` fields) rather than guessing at safe defaults.